

“Linux System Information and Resource Monitoring Tool”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

G. AKSHAY

2520030363

B. ARYAN

2520030343

TEJ PRATAP SINGH

2520030476

Under the Esteemed Guidance of

[TO BE ADDED]

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled “Linux System Information and Resource Monitoring Tool” is a record of bona fide work carried out by G. AKSHAY - 2520030363, B. ARYAN - 2520030343, and TEJ PRATAP SINGH - 2520030476 and submitted in partial fulfillment of the requirements for the award of the B.Tech degree in Computer Science and Engineering at K L (Deemed to be) University. The work described in this report is based on the project developed for the Operating Systems and Systems Programming course. To the best of our knowledge, the results and implementation presented in this report have not been copied from any other department, university, or institute.

G. AKSHAY	2520030363
B. ARYAN	2520030343
TEJ PRATAP SINGH	2520030476

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini-project report entitled “Linux System Information and Resource Monitoring Tool” is a bona fide work done and submitted by G. AKSHAY - 2520030363, B. ARYAN - 2520030343, and TEJ PRATAP SINGH - 2520030476,

in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering, K L (Deemed to be) University, during the academic year 2026-2027.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project was possible because of the guidance, facilities, technical support, and encouragement received throughout the development and evaluation process. We sincerely thank everyone who directly or indirectly contributed to the completion of this Operating Systems and Systems Programming project.

We express our sincere gratitude to our Course Coordinator, [TO BE ADDED], and our project guide, [TO BE ADDED], Department of Computer Science and Engineering, for their guidance, review suggestions, and encouragement during the planning, implementation, testing, and documentation of the project.

We also thank the Head of the Department, [TO BE ADDED], and the Department of Computer Science Engineering for providing the academic environment and facilities required to carry out this project-based work.

We thank all teaching and non-teaching staff members who supported us during the course. Finally, we sincerely thank our parents, friends, and classmates for their cooperation and encouragement throughout the development, debugging, testing, and presentation of this project.

G. AKSHAY 2520030363

B. ARYAN 2520030343

TEJ PRATAP SINGH 2520030476

Abstract

Modern operating systems expose a large amount of runtime information about processors, memory, processes, storage, networking, and system state, but this information is distributed across multiple Linux interfaces and commands. The project “Linux System Information and Resource Monitoring Tool” was developed to provide a unified and educational view of this information while directly demonstrating concepts from Operating Systems and Systems Programming. The project uses C and POSIX programming to collect live system data from Linux virtual filesystems such as /proc and /sys, and from operating-system APIs such as statvfs(), gethostname(), getifaddrs(), POSIX threads, signals, pipes, process-control functions, and sockets.

The system is implemented as a real monitoring core with both terminal and web-based interfaces. It reports CPU utilization, load averages, memory and swap usage, root-filesystem usage, uptime, network receive/transmit activity, process counts, and detailed per-process information including PID, PPID, user, state, CPU percentage, RAM usage, thread count, command line, and start time. A process inspector exposes additional information from /proc/<PID>, while lifecycle tracking records STARTED, ACTIVE, and EXITED events using the combination of PID and process start time so that PID reuse does not corrupt the history. A controlled process/IPC demonstration uses pipe(), fork(), a child process, and waitpid() to show core OS concepts in a repeatable manner.

The project also supports authorized agentless monitoring of another Linux or WSL machine over OpenSSH. The browser presents an IP-only quick-connect interface, while authentication remains an operating-system responsibility through configured SSH keys and backend settings; passwords and private keys are never collected through the browser. In WSL, Linux /proc data continues to represent the Linux environment, while Windows-host CPU/uptime and visible Windows applications can be obtained separately through controlled PowerShell interop. The completed implementation includes a C socket HTTP server, JSON APIs, a responsive HTML/CSS/JavaScript dashboard, source-isolation safeguards, input validation, automated tests, and clean signal-driven shutdown. The project therefore combines practical system monitoring with direct exposure to user/kernel interaction, process management, memory management, file I/O, concurrency, IPC, networking, and secure remote administration principles.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–3
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		4–7
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		8–9
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		10–12
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		13–16
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		17–22
		6.1 Module-wise Implementation	

		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		23–26
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		27–28
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		29
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

This project was developed for the course Operating Systems and Systems Programming (25CS2104E), 2026-27 Term-I, Batch-7, Team 7. Operating systems continuously maintain information about processor activity, memory allocation, process state, file systems, devices, and network interfaces. In Linux, much of this information is exposed through virtual kernel interfaces rather than through a single application programming interface. The `/proc` filesystem provides process and kernel statistics, while `/sys` exposes device and subsystem information. Applications can also obtain information through POSIX and Linux APIs such as `statvfs()`, `gethostname()`, `getifaddrs()`, `socket()`, `open()`, `read()`, `close()`, `fork()`, `waitpid()`, and `pthread` functions. These mechanisms are important because they form the bridge between a user-space program and the services maintained by the kernel.

The idea behind this project is to convert those low-level operating-system sources into a coherent monitoring application without hiding the underlying concepts. Instead of using a high-level monitoring library or scraping the output of an existing program, the monitoring core reads the Linux interfaces directly using C. The same collected snapshot is then made available through a terminal interface and through a lightweight web dashboard. This design keeps the project closely aligned with the objectives of Operating Systems and Systems Programming while also producing an interface that is easy to demonstrate during a laboratory review.

The project evolved from a basic system-information utility into a broader observability tool. The final design includes system telemetry, a live process table, per-PID inspection, process lifecycle tracking, an open-applications view, a controlled process/IPC demonstration, a C-based HTTP/JSON layer, and authorized remote monitoring. The remote feature is deliberately agentless: the complete project does not need to be copied to the second laptop. Instead, the main system uses the local OpenSSH client to execute a fixed read-only collection script on an already authorized Linux or WSL machine and normalizes the result into the same snapshot format used for local monitoring.

Because the project is frequently demonstrated from Windows laptops using Ubuntu under WSL2, the implementation also distinguishes between the Linux environment and the Windows host. Linux process information is correctly obtained from WSL `/proc` and is not falsely presented as the complete Windows process list. When WSL is detected, controlled Windows PowerShell interop may be used for host-level CPU/uptime measurements and for visible native Windows applications such as Chrome, Edge, VS Code, File Explorer, and similar programs. This hybrid behavior is kept explicitly separate from the Linux PID table so that the data source remains technically accurate.

1.2 Problem Statement

Linux provides rich system information, but the information required to understand overall resource usage is distributed across multiple virtual files, directories, system calls, and command-line utilities. A student who runs `top`, `ps`, `free`, `df`, `uptime`, or `ip` can view useful output, but these tools do not necessarily explain how the values are obtained or how operating-system concepts such as process identity, parent-child relationships, lifecycle transitions, jiffies, file descriptors, virtual memory, synchronization, and signal handling are connected. This makes it difficult to use a single demonstration to explain both the theory and the actual operating-system mechanisms.

Another difficulty is process observation. A simple process list only shows what exists at the instant of the command. Short-lived processes may appear and disappear between samples, and PID values may later be reused by the kernel. Therefore, a useful educational monitor must do more than display a current PID list: it should compare consecutive snapshots, distinguish one process instance from another using PID plus start time, and record meaningful lifecycle events such as `STARTED`, `ACTIVE`, and `EXITED`.

Remote monitoring introduces an additional problem. The project should be able to inspect an authorized second Linux/WSL laptop without copying the entire project or creating an unsafe browser-controlled remote shell. The connection must respect normal SSH authentication and must not accept arbitrary command text, passwords, or private keys from the web interface. At the same time, a source-selection failure must never cause local measurements to be mislabeled as remote data. These requirements led to a design that combines direct kernel-data collection, controlled concurrency, source isolation, and fixed SSH execution paths.

1.3 Scope of the Project

The scope of the project covers live monitoring of a Linux environment and, when configured, an authorized remote Linux/WSL environment. The system collects CPU usage, CPU model and logical-core count, load averages, physical memory usage, swap usage, root-filesystem capacity, system uptime, hostname, distribution, kernel version, process counts, thread count, and network-interface statistics. Network information includes interface name, link state, IPv4 address, cumulative received/transmitted byte counters, and interval-based receive/transmit rates.

At the process level, the monitor scans numeric /proc directories and constructs a bounded process snapshot containing PID, PPID, UID/user, process name, state, CPU percentage, RAM usage, thread count, command line, and process start time. A selected PID can be inspected for executable path, working directory, virtual/resident memory information, I/O counters, open-file-descriptor count, and memory maps when permissions allow. A separate Open Applications layer converts low-level process information into a more understandable view of GUI applications without replacing the raw process table.

The project also includes a controlled Systems Lab / Process-IPC demonstration. It creates an anonymous pipe, forks a child, has the child read /proc/uptime, sends that result through the pipe, and allows the parent to read the message and call waitpid(). This module exists to demonstrate real process creation, IPC, synchronization, and termination without exposing arbitrary user commands. The web interface displays the result of the same fixed demonstration through a JSON endpoint.

The project is not designed as an enterprise monitoring platform, a Windows kernel monitor, a vulnerability scanner, a credential-capture system, or a stealth administrative agent. It does not perform random-host scanning, privilege escalation, hidden persistence, or browser-based remote shell execution. Remote monitoring is limited to hosts that the user has already authorized through ordinary OpenSSH configuration.

1.4 Objectives of the Project

The primary objective is to design and implement a Linux system information and resource monitoring tool that demonstrates operating-system concepts through a practical, observable application. The project objectives are summarized below and are implemented as one coherent system rather than as isolated laboratory programs.

Table 1.1 – Project objectives and implementation mapping

Objective	Project realization
Collect real Linux telemetry	Read CPU, memory, uptime, process, disk, load and network data from /proc, /sys and OS APIs.
Monitor processes	Display PID, PPID, user, state, CPU, memory, threads, command line and start time.
Explain process lifecycle	Track STARTED, ACTIVE and EXITED events using PID + start time to handle PID reuse.
Demonstrate system programming	Use C, POSIX file I/O, processes, threads, mutexes, signals, sockets, pipes and waitpid().

Objective	Project realization
Provide detailed PID inspection	Expose <code>/proc/<PID></code> status, I/O, file-descriptor and memory-map information when permitted.
Provide a usable interface	Offer both terminal output and a page-based web dashboard called KernelScope.
Support authorized remote observation	Monitor a configured Linux/WSL host over OpenSSH without installing the project on the remote laptop.
Protect source correctness	Clearly label local/remote status and avoid presenting stale local values as remote data.
Support WSL demonstrations	Keep WSL Linux processes separate while optionally showing Windows-host CPU/uptime and visible applications.

1.5 Importance of the Application

The application is important from an educational perspective because it turns abstract operating-system concepts into observable runtime behavior. For example, the relationship between user space and the kernel becomes visible when the C program reads `/proc/stat`, `/proc/meminfo`, or `/proc/<PID>/status`. The role of sampling is visible in CPU calculations; the role of process identity is visible in PID/PPID and start time; concurrency becomes necessary when the monitoring collector and web clients operate at the same time; and synchronization becomes concrete when multiple threads access the shared snapshot.

The project is also useful as a lightweight practical monitor. Instead of opening several commands, a student can use one dashboard to view the overall system, drill down into process details, inspect network activity, review lifecycle history, and switch to an authorized remote source. The dashboard improves readability, but it does not replace the low-level core; the web layer simply visualizes information produced by the C monitoring engine.

A further educational benefit is the separation of data sources in WSL. Windows-host applications are not Linux `/proc` processes, and the project does not pretend otherwise. Showing Windows visible applications through a separate interop path while keeping the Linux process table based on WSL `/proc` demonstrates an important systems principle: a measurement is only meaningful when its source and environment are correctly identified.

1.6 Target Users / Audience

The primary users are B.Tech students studying Operating Systems and Systems Programming, faculty members evaluating laboratory/project work, and learners who want to understand how Linux exposes resource and process information. The terminal mode is useful when the focus is on source code and kernel interfaces, while the web mode is useful during demonstrations because it groups system metrics, process information, lifecycle events, and remote-machine state into a single interface.

The application can also be used by developers or learners who need a small read-only observability tool for their own Linux or WSL environments. Remote functionality is intended only for machines for which the user already has authorization. The project is not intended for monitoring arbitrary third-party systems.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project is intentionally structured around the operating-system concepts taught in the course. The program itself runs in user space, but it obtains data maintained by the Linux kernel through virtual filesystems and system APIs. This illustrates the service-layer model of an operating system: the user-space program does not directly access hardware counters or kernel structures. Instead, it uses controlled interfaces such as files under `/proc` and `/sys` or functions implemented through system calls and standard libraries.

Process management is central to the design. Every process has a PID and, except for the root of the process hierarchy, a PPID. The monitor reads process state and timing information from `/proc/<PID>/stat`, ownership and thread data from `/proc/<PID>/status`, and command-line data from `/proc/<PID>/cmdline`. Because the kernel may reuse PID values, the project identifies a specific process instance using both PID and the start-time field from `/proc/<PID>/stat`. This prevents a newly created process from being incorrectly treated as the continuation of an older process that previously had the same PID.

CPU utilization requires interval measurement rather than a single counter. Linux exposes cumulative CPU jiffies in `/proc/stat`. The monitor stores one snapshot and compares it with the next. If `Total` represents the sum of aggregate CPU time fields and `Idle` represents idle plus appropriate idle time, the interval utilization is calculated as $\text{CPU}\% = 100 \times (\Delta\text{Total} - \Delta\text{Idle}) / \Delta\text{Total}$. The same principle is used for per-process CPU utilization by comparing the change in `utime` + `stime` with the change in aggregate system jiffies and scaling by the number of online logical CPUs.

Memory management is represented through `/proc/meminfo` and per-process status fields. The project distinguishes total, available, used, and swap memory for the system, while per-process memory is represented in RAM MB and percentage values. The process inspector can additionally expose virtual and resident memory information. These values demonstrate the difference between a process address space and the amount of physical memory currently resident for the process.

File I/O concepts appear throughout the project. The core uses bounded reads, directory scans, `open()/read()/close()`, `readlink()`, and filesystem APIs. The process inspector reads `/proc/<PID>/io`, counts entries under `/proc/<PID>/fd`, and can return `/proc/<PID>/maps`. Root-filesystem usage is obtained through `statvfs()`. These operations connect the abstract concepts of files, file descriptors, and virtual filesystems to real operating-system measurements.

Concurrency is required because continuous collection should not block the user interface. A long-lived POSIX monitor thread periodically creates a new snapshot. The HTTP server creates short-lived detached worker threads for accepted clients. Shared snapshot, lifecycle, and source-selection state is protected by a pthread mutex. This provides a concrete example of race-condition prevention: readers should not observe partially updated structures while the collector is replacing them.

IPC and process control are demonstrated through the process-demo module. The program creates an anonymous pipe, calls `fork()`, performs different work in the parent and child, passes a real `/proc/uptime` value from child to parent, and synchronizes with `waitpid()`. Signal handling is implemented with `sigaction()` for `SIGINT` and `SIGTERM`, allowing the monitor to stop its loops and release resources in a controlled way instead of terminating abruptly.

Networking appears at two levels. Locally, network-interface counters are read from `/sys/class/net` and interface addressing is obtained through system APIs. At the application layer, the web server directly uses POSIX sockets: `socket()`, `setsockopt()`, `bind()`, `listen()`, `accept()`, `recv()`, and `write/send` behavior. Remote monitoring additionally launches the local OpenSSH client through `fork()/exec` rather than

constructing shell commands from browser input.

Table 2.1 – Mapping of operating-system concepts to implementation

OS concept	Where it appears in the project
User/kernel space	User-space C program reads kernel-maintained /proc and /sys interfaces.
System calls / file I/O	open, read, close, readlink, directory scans and filesystem queries.
Processes	PID, PPID, states, fork, waitpid, process inspection and lifecycle history.
IPC	Anonymous pipe used by the process demonstration.
Signals	sigaction for controlled SIGINT/SIGTERM shutdown.
Memory management	System memory/swap and per-process virtual/resident memory fields.
Concurrency	Collector thread, HTTP worker threads, mutex-protected shared state.
Networking	/sys counters, getifaddrs, POSIX HTTP sockets and SSH-based remote collection.

2.2 Existing Methods / Algorithms

Linux already provides mature utilities for system inspection. top provides an interactive process and resource view; ps reports process information; free summarizes memory; df reports filesystem usage; uptime provides load averages and uptime; and ip exposes interface configuration. These tools are effective for administration, but the educational objective of this project is different. Calling a sequence of existing commands would demonstrate command usage rather than demonstrate how a C program obtains the underlying information.

A second approach would be to use a high-level monitoring library. Such a library can simplify CPU, memory, and process collection, but it also hides the implementation details that are important for this course. For that reason, the project does not use a high-level monitoring framework as the source of its telemetry. It uses the Linux interfaces directly and implements the calculations and snapshot comparison itself.

Traditional monitoring systems may also deploy an agent on every remote machine. An agent can provide richer capabilities and persistent connectivity, but it would increase setup requirements for a college demonstration. The proposed project instead uses an agentless model for the remote feature. The remote system only needs normal Linux/WSL and an OpenSSH server; a fixed read-only script is sent through an already authorized SSH session.

For lifecycle observation, a simple algorithm can compare only PID values between snapshots. That approach is incorrect when PID reuse occurs. The project therefore uses a stronger identity key composed of PID and process start ticks. This is a small but important design decision because it aligns the software with actual Linux process semantics.

Table 2.2 – Background comparison of common approaches

Existing approach	Strength	Limitation for this project
top / htop	Excellent interactive monitoring	Does not directly demonstrate project-owned data collection and algorithms.
ps, free, df, uptime, ip	Simple and widely available	Information is fragmented across multiple commands.

Existing approach	Strength	Limitation for this project
High-level monitoring library	Fast application development	Hides /proc, /sys, jiffies, process parsing and lower-level APIs.
Remote agent installation	Can provide persistent rich monitoring	Requires software installation/management on every monitored laptop.
PID-only lifecycle matching	Easy to implement	Can misidentify a new process when Linux reuses a PID.

2.3 Proposed Approach

The proposed approach uses a reusable C monitoring core that periodically collects a complete snapshot of the selected source. For local Linux monitoring, the collector reads /proc, /sys, and operating-system APIs directly. It then calculates interval-dependent measurements, scans processes, builds process objects, updates lifecycle state, and publishes the snapshot under mutex protection. Both the terminal program and the web program use this same monitoring core, which avoids having separate implementations with inconsistent values.

The web program adds a small HTTP server written using POSIX sockets. It serves static HTML/CSS/JavaScript assets and exposes only the JSON API routes needed by the dashboard. The browser therefore remains a visualization client; it does not execute Linux commands. API responses describe the active source, freshness, metrics, process rows, lifecycle events, network information, and application information. This separation allows the backend to enforce validation and source isolation centrally.

For remote mode, the selected machine is represented by a validated configuration. The backend invokes OpenSSH with a fixed argument list and sends a fixed read-only monitoring script to remote sh through standard input. The remote script reads kernel interfaces and returns normalized records. The local C backend parses those records, performs interval calculations, and converts them to the same MonitorSnapshot structure used locally. This means the frontend does not need separate rendering logic for local and remote telemetry.

For WSL, the project deliberately implements a hybrid but labeled model. Linux resources and processes are still read from the WSL kernel environment. Windows-host CPU/uptime and visible native Windows applications can be collected through a fixed PowerShell query invoked via WSL interop. The resulting rows are labeled as Windows-host data and are not merged into the Linux process table. In remote WSL mode, the same concept is implemented through the authorized SSH session if PowerShell interop is available on the remote WSL installation.

2.4 Advantages of the Proposed Approach

The proposed design is closely aligned with systems-programming learning outcomes because the important measurements are not hidden behind a monitoring framework. Students can trace a value from the dashboard back to a JSON field, a C structure, a collector function, and finally a Linux kernel interface. The same project therefore supports both demonstration and viva-level explanation.

The architecture also improves correctness. Snapshot-based collection gives a consistent set of values for one sampling point. Mutex protection prevents races between collection and HTTP readers. PID plus start time prevents lifecycle corruption caused by PID reuse. Remote source selection and unavailable-state handling prevent local values from being mislabeled as remote measurements. Input validation and fixed exec-based SSH invocation reduce the possibility of shell-command injection.

The tool remains lightweight because no database, application framework, Node.js package, Python

web framework, or remote agent is required. A normal GCC/POSIX environment is sufficient to build the binaries, and the static dashboard is served directly by the C HTTP server. The result is small enough to explain file by file while still including features that make a final project demonstration visually strong.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The project does not depend on specialized hardware. It is intended to run on an ordinary laptop or desktop capable of running Linux directly or Ubuntu under Windows Subsystem for Linux. Because the project information did not define mandatory numeric minimums for processor, RAM, or disk capacity, the documentation does not invent such limits. The practical requirements are therefore expressed functionally below.

Table 3.1 – Hardware requirements

Hardware item	Requirement
Processor	Any processor supported by the chosen Linux/Ubuntu environment. Multi-core CPUs allow more interesting utilization demonstrations.
Memory	Sufficient RAM to run the operating system, compiler, web browser, and monitor. [TO BE ADDED: college-required numeric minimum, if any].
Storage	Sufficient free space for GCC/build tools and the project source. [TO BE ADDED: college-required numeric minimum, if any].
Display	Required for the KernelScope web dashboard; not required for terminal-only mode.
Network adapter	Required only for remote-laptop demonstrations. Local monitoring works without a network connection.
Second laptop	Optional. Needed only to demonstrate physical remote monitoring; not required for local functionality.

3.2 Software Requirements

Table 3.2 – Software requirements

Software	Purpose / requirement
Linux / Ubuntu	Primary runtime environment. The core expects Linux /proc and /sys interfaces.
WSL2 (optional)	Allows the Linux project to run on Windows while the dashboard is opened from a Windows browser.
GCC	Compiles the C11/POSIX source code.
make	Builds system_monitor and monitor_web using the supplied Makefile.
OpenSSH client	Required on the main machine for remote monitoring.
OpenSSH server	Required on an authorized remote Linux/WSL laptop for remote monitoring.
Modern browser	Displays the HTML/CSS/JavaScript dashboard at localhost:8080.
PowerShell interop (WSL feature)	Optional path used to read Windows-host CPU/uptime and visible native Windows applications when available.
curl	Useful for API testing and demonstration; not required for normal dashboard use.

No npm, Node.js application server, Flask, Django, Spring Boot, database server, or third-party

JavaScript package is required by the final project. The web assets are static files and are served by the C socket server. This keeps installation small and ensures that the main logic remains in C rather than being replaced by a high-level framework.

3.3 Operating System Environment – Ubuntu / WSL

The core program requires a Linux environment because its primary data sources are Linux virtual filesystems. On a native Ubuntu installation, /proc and /sys represent the physical Linux operating system. On WSL2, they represent the Linux kernel environment associated with the WSL distribution. This distinction is reflected in the user interface and documentation so that Linux process values are not described as complete Windows-host process values.

When the project runs inside WSL2, the Windows browser can normally access the server through `http://localhost:8080`. This provides a convenient development workflow: compilation and system programming remain inside Ubuntu/WSL, while the graphical dashboard can be opened in Chrome or Edge on Windows. Windows-host application enumeration is optional and uses a fixed PowerShell query through WSL interop. If interop is unavailable, the project falls back to Linux GUI-session detection rather than fabricating Windows application rows.

For remote monitoring of another WSL machine, the remote WSL distribution must be running and its SSH service must be reachable. Closing a terminal window does not automatically prove that WSL has stopped; the correct test is whether the SSH service and distribution remain active. If the remote Linux environment stops, the dashboard must report the remote source as unavailable/disconnected rather than silently displaying local measurements.

3.4 Development Tools and Compilers

The project is compiled with GCC in C11 mode and uses the POSIX thread library. The Makefile applies -Wall, -Wextra, and -Wpedantic warning options in addition to optimization and pthread flags. A debug target is also available with AddressSanitizer and UndefinedBehaviorSanitizer. This makes compiler warnings and runtime diagnostics part of the development process rather than relying only on visual testing.

The project can be edited in Visual Studio Code or any text editor, while the actual build and execution commands are run in the Ubuntu/WSL terminal. Git and GitHub are used for source management and submission. The final repository contains the project under a Project folder together with README/start instructions and the complete source structure.

Typical Ubuntu/WSL setup and build commands

```
sudo apt update
sudo apt install build-essential curl openssh-client -y

make clean
make
./monitor_web
```

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project is implemented in C. The template heading mentions C/C++, but C++ is not used in the final implementation. C was selected because it provides direct access to POSIX APIs and maps naturally to the systems-programming concepts studied in the course. Structures are used to represent system information, process information, monitor snapshots, lifecycle records, machine configurations, API responses, and server state.

Dynamic memory is used where variable-sized JSON output or file content is required, while large bounded arrays are used for process and lifecycle snapshots. Error paths are checked explicitly, file reads are bounded, and operating-system return codes are converted into usable error messages. This programming style is important for systems work because failures such as unreadable /proc files, disappearing processes, bind conflicts, and unavailable SSH connections are expected runtime conditions rather than exceptional theoretical cases.

4.2 GCC Compiler

GCC compiles both the terminal and web binaries. The release build uses the flags shown below. These flags enforce the C11 language mode, enable common warning classes, compile with optimization, link POSIX threads, and include project header files. The final release build was executed with make clean followed by make and completed without compiler warnings under these configured warning options.

Representative Makefile configuration

```
CC := gcc
CFLAGS := -std=c11 -Wall -Wextra -Wpedantic -O2 -pthread -Iinclude
LDFLAGS := -pthread

all: system_monitor monitor_web
```

The debug target uses compiler sanitizers so that memory-access mistakes and undefined behavior can be detected during testing. This was used to exercise the main terminal, API, application, process inspection, and shutdown paths.

4.3 Linux System Calls

Linux and POSIX system interfaces are used throughout the project. For telemetry, files under /proc and /sys are opened and read like files even though many of them are dynamically generated kernel interfaces. For process control, fork() creates a child process and waitpid() synchronizes the parent. For IPC, pipe() creates a unidirectional communication channel. For signal handling, sigaction() installs controlled shutdown handlers.

The HTTP server uses the socket API directly. socket() creates the endpoint, setsockopt() applies socket behavior, bind() attaches it to a local address and port, listen() creates the accept queue, and accept() returns a connected client descriptor. The server then receives a bounded request, routes it to the API layer or static-file handler, writes the response, and closes the connection. This implementation is intentionally small and project-specific rather than a general web framework.

Table 4.1 – Important system/POSIX interfaces

Call / API	Use in the project
open(), read(), close()	Read virtual files and serve static web assets.

Call / API	Use in the project
opendir(), readdir()	Scan numeric /proc directories and file-descriptor directories.
readlink()	Resolve process executable and working-directory links.
fork()	Create child process for IPC demo and controlled external program execution.
pipe(), dup2()	IPC demo and capturing output from fixed child processes.
waitpid()	Synchronize/reap child processes and avoid zombies.
sigaction()	Install SIGINT/SIGTERM clean-shutdown handlers.
socket(), bind(), listen(), accept()	Implement the C HTTP server.
statvfs()	Read filesystem capacity and usage.
gethostname(), uname()	Obtain machine and kernel information.
getifaddrs()	Associate interface information with IPv4 addresses.

4.4 POSIX Threads / Synchronization

The monitor starts a long-lived collector thread using `pthread_create()`. The worker wakes according to the configured sampling interval, collects a local or remote snapshot, updates lifecycle state, and publishes the result. The HTTP server independently creates detached worker threads for accepted clients so that one slow request does not stop the server from accepting every other request.

Shared state is protected with `pthread_mutex_t`. A mutex guards snapshot data, the selected source, lifecycle history, tracked-process state, and related monitor status. API readers copy the current snapshot while the mutex is held briefly and perform JSON formatting outside the critical section when possible. This reduces contention and prevents readers from seeing partially modified state.

4.5 Linux Commands and Utilities

Commands such as `make`, `gcc`, `curl`, `ssh`, `ssh-keygen`, `ssh-copy-id`, and `systemctl/service` are used for building, testing, or configuring the environment. The monitoring data itself is not obtained by repeatedly running `top` or `ps`. Direct kernel interfaces remain the authoritative source for the core local process and resource data.

For remote monitoring, the command-line `ssh` client is intentionally used as an operating-system component rather than implementing the SSH protocol inside the project. Authentication is configured using ordinary OpenSSH mechanisms. The browser only supplies an IPv4 address to the quick-connect endpoint; it does not supply a password or private key.

Authorized SSH preparation used for the remote feature

```
# Main laptop / WSL
ssh-keygen -t ed25519
ssh-copy-id remote_user@REMOTE_IP
ssh -o BatchMode=yes remote_user@REMOTE_IP "hostname && cat /proc/uptime"

# Remote Linux / WSL
sudo systemctl enable --now ssh
```

4.6 Other Tools / Libraries Used

The frontend uses standard HTML, CSS, and browser JavaScript. JavaScript periodically calls the JSON API and updates the page without requiring a frontend framework. The interface is organized into

Overview, Processes, Lifecycle, Network, Systems Lab, and Machines pages. This page-based layout improves demonstration clarity while keeping deployment dependency-free.

OpenSSH is used for remote transport, and optional Windows PowerShell interop is used only when WSL is detected and the relevant functionality is available. The remote Open Applications probe can also use PowerShell through remote WSL so that visible Windows-host applications can be shown separately from the remote Linux process table. All PowerShell commands used by the project are fixed in source; browser-controlled text is not concatenated into the script.

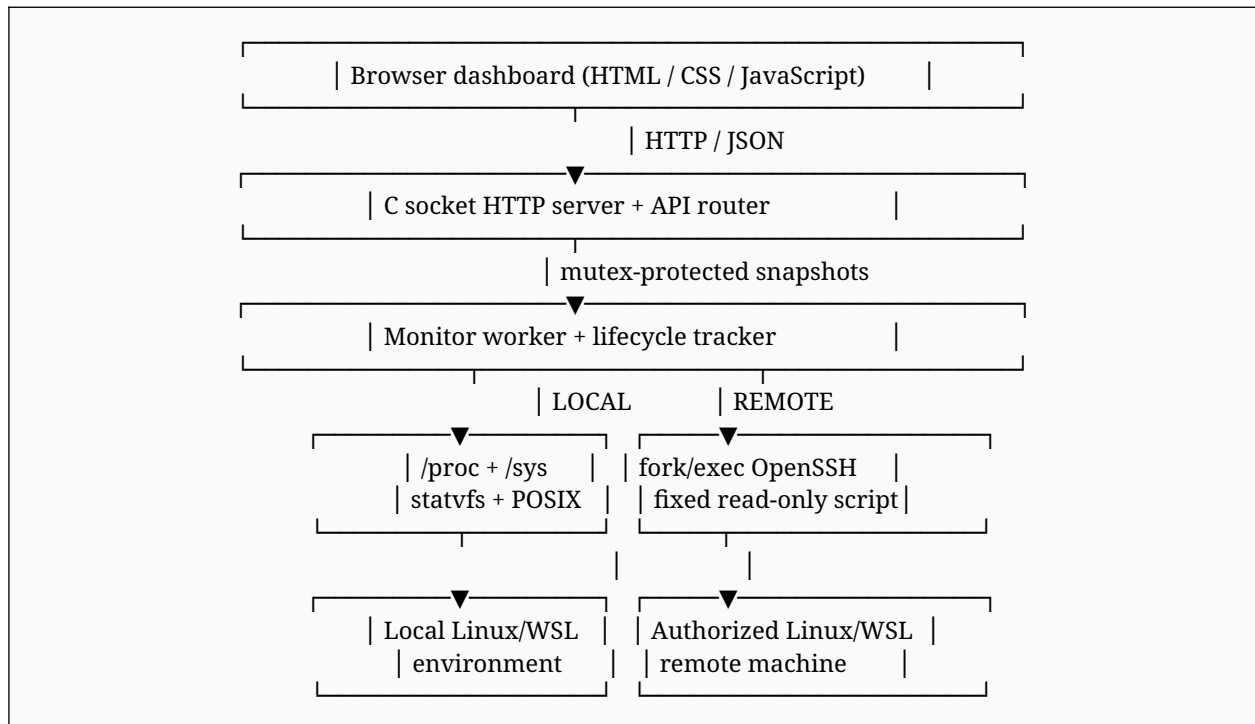
GitHub is used to store the final project source and documentation. The submitted repository is: https://github.com/Tejpratap-069/KLH-CSE-2026-2027-Team-7-Linux_System_Information_and_Resource_Monitoring_Tool/tree/main/Project .

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The design follows a layered architecture. The lowest layer consists of Linux kernel interfaces and operating-system APIs. Above that is the C monitoring core, which normalizes raw information into a MonitorSnapshot. The API/server layer exposes the current snapshot and process details as JSON. Finally, the terminal interface or browser dashboard presents the information. Remote monitoring replaces the local collector with an SSH-backed collector but preserves the same normalized snapshot model.

Figure 5.1 – Overall system architecture



The MonitorSnapshot structure is a key design element because it gives local and remote collectors the same output contract. It contains source metadata, machine identity, timestamp/freshness information, system measurements, and a bounded process array. This prevents the frontend from depending on collector-specific implementation details.

The source selector is also explicit. The selected source can be local or a saved remote machine. When the user switches machines, lifecycle tracking is reset for the new source so that events from two machines are not combined. If remote collection fails before a valid remote snapshot is available, the API reports an unavailable condition rather than returning a local snapshot under a remote label.

5.2 System Workflow

When monitor_web starts, it initializes machine configuration, the monitor context, the API context, signal handlers, and the C socket server. The monitor collector thread begins sampling the selected source. In local mode, the worker reads system counters and then scans /proc for processes. In remote mode, it calls the registered remote collector, which invokes OpenSSH and parses the returned normalized records.

After a snapshot has been collected, the lifecycle tracker compares the new process set with the bounded tracked-process table. New (PID, starttime) pairs create STARTED events, continuing pairs

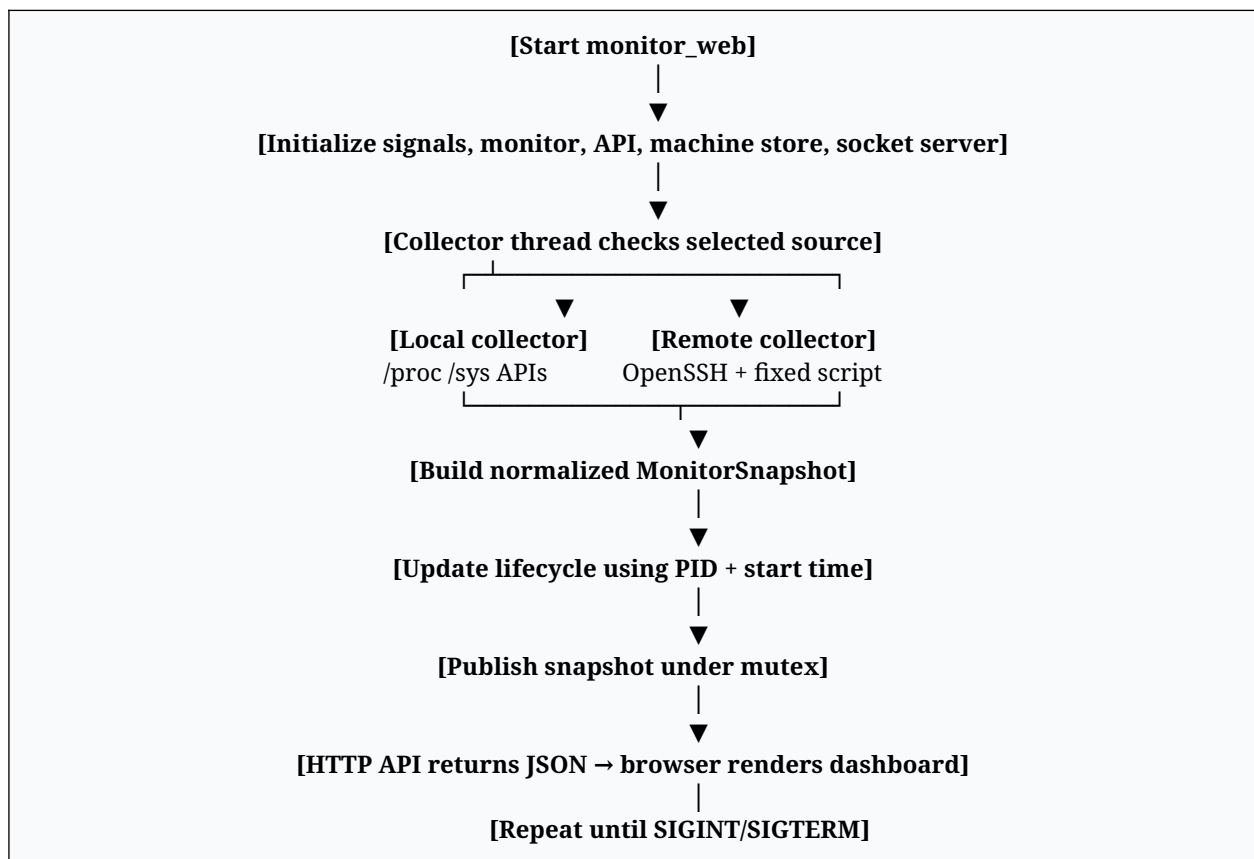
update latest and peak resource values and may generate ACTIVE events, and previously tracked pairs that disappear create EXITED events. The new snapshot is then published under the monitor mutex.

The browser periodically calls API endpoints such as /api/system, /api/processes, /api/lifecycle, /api/network, and /api/applications. The API copies the current stable snapshot, formats it as JSON, and returns it. The browser updates cards, tables, status labels, and charts. Process-inspector requests are performed only when the user selects a specific PID, which avoids performing expensive detailed reads for every process on every refresh.

Remote quick connect begins with an IPv4 address in the browser. The backend validates the address, applies backend-only SSH defaults, tests the authorized connection, saves the endpoint if required, activates the machine, and changes the monitoring source. Authentication remains with OpenSSH; the web form does not receive a password or key.

5.3 Flowchart / Data Flow

Figure 5.2 – Monitoring and presentation data flow



The flowchart shows that the browser is not part of the measurement path. The operating-system data is collected first, normalized in the C backend, and only then delivered to the web interface. This design is important for correctness and security because the browser does not directly execute system commands.

5.4 Module Description

Table 5.1 – Major source modules

Module / files	Responsibility
src/system_info.c	Collect CPU, memory, load, uptime, disk, hostname, kernel and network/system information; includes WSL host CPU/uptime bridge.

Module / files	Responsibility
src/process_monitor.c	Parse /proc process data, calculate per-process CPU/RAM, inspect a selected PID and expose memory maps.
src/monitor.c	Run the collector thread, manage snapshots, source selection and process lifecycle history.
src/application_monitor.c	Build Open Applications view for native Linux and WSL Windows-host visible applications.
src/process_demo.c	Perform controlled pipe + fork + child message + waitpid demonstration.
src/terminal_main.c	Terminal-oriented execution and one-shot/process-demo modes.
backend/web_server.c	Implement bounded HTTP server using POSIX sockets and worker threads.
backend/api.c	Route JSON API requests, machine selection, quick connect, process/application/lifecycle endpoints.
backend/remote_ssh.c	Execute fixed SSH collection/inspection scripts and parse remote Linux/WSL data.
backend/machine_store.c	Validate, save, list and select local/remote machine configuration.
frontend/index.html	Dashboard page structure.
frontend/styles.css	KernelScope visual theme and responsive layout.
frontend/app.js	API polling, source switching, tables, modal controls and client-side rendering.
tests/*.sh	Smoke, API, process, lifecycle, applications, security, source-state and quick-connect validation.

5.5 Input and Output Design

The normal local-monitoring path requires no user input. The collector continuously reads the running operating system and the dashboard displays the result. User inputs are limited to actions such as selecting a PID, starting the fixed process demonstration, changing the monitored machine, or entering an IPv4 address in the remote quick-connect dialog.

Remote input is intentionally restricted. The quick-connect form asks for an IPv4 address only. The backend validates the value before any SSH call. Username, identity file, and optional port are backend configuration concerns and can be resolved through OpenSSH configuration or environment variables such as `MONITOR_SSH_USER` and `MONITOR_SSH_IDENTITY`. This prevents the browser from becoming a credential-collection surface.

Outputs are provided in two forms. The terminal binary prints system/process information directly, while the web binary returns JSON and serves a browser interface. The dashboard uses clear source labels such as `LOCAL — LIVE`, `REMOTE — LIVE`, or an unavailable/disconnected status. Numeric metrics are displayed with appropriate units and process rows remain traceable to real PIDs. When an operating-system permission prevents reading a field, the project reports unavailable information rather than crashing or inventing a value.

Table 5.2 – Core HTTP/JSON interfaces

API route	Purpose
GET /api/health	Server health check.

API route	Purpose
GET /api/system	Current system snapshot and source status.
GET /api/processes	Current process list.
GET /api/process/<pid>	Detailed process inspection.
GET /api/process/<pid>/maps	Process memory-map text when accessible.
GET /api/lifecycle	Recent STARTED/ACTIVE/EXITED lifecycle events.
GET /api/network	Current network-related snapshot data.
GET /api/applications	Open Applications view for active local/remote source.
POST /api/process-demo	Runs the fixed process/IPC demonstration.
GET /api/machines	Lists local and saved remote sources.
POST /api/quick-connect	Validates/tests/selects an IP-only remote source.
POST /api/select-machine	Switches the active monitoring source.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

System information collection

System collection begins with aggregate CPU counters from `/proc/stat`. The collector stores total and idle tick values in the snapshot so that the next collection can calculate an interval percentage. CPU model and logical-core information are obtained from `/proc/cpuinfo`. Distribution information is read from the Linux release information available in the system, while kernel information is obtained from `uname()`. Uptime comes from `/proc/uptime` and load averages are read from `/proc/loadavg`.

Memory values are read from `/proc/meminfo` and converted into total, available, used, and swap fields. Root-filesystem usage is collected using `statvfs()`, which avoids parsing the text output of `df`. Network interfaces are enumerated and correlated with `/sys/class/net` counters and IPv4 addresses. RX/TX rates are calculated from byte-counter differences and the interval between snapshots.

Process collection and inspection

The process collector scans `/proc` and selects directory names that contain only digits. It parses `/proc/<PID>/stat` carefully because the process command name is enclosed in parentheses and may contain characters that make simple whitespace tokenization unreliable. `/proc/<PID>/status` provides UID, memory, and thread data, while `/proc/<PID>/cmdline` provides the command line. The collector maps UID values to user names when possible and formats the process start time using uptime and clock-tick information.

Per-process CPU utilization is calculated only after a previous snapshot exists. A previous process is matched by PID and start ticks. The collector compares `utime + stime` with the previous process ticks, divides by the aggregate system-jiffy delta, multiplies by the number of logical CPUs, and scales by 100. Consequently, approximately one fully occupied core appears around 100% even on a multi-core machine.

The PID inspector performs additional reads only for the selected process. It exposes executable and working-directory links, selected status fields, I/O counters, file-descriptor count, and memory maps. Linux permissions are respected: inaccessible information is returned as unavailable instead of using elevated privileges.

Monitoring worker and lifecycle tracker

`monitor.c` owns the long-lived worker thread. The thread copies the selected-source configuration under the mutex, performs collection outside the critical section, and then reacquires the mutex to publish the result. This keeps the critical section short while ensuring that the active source cannot silently contaminate another source. A source-generation value is used so that results from an outdated collection attempt can be discarded when the user changes machines during collection.

The lifecycle tracker maintains a bounded table of tracked process instances. Every new snapshot marks matching entries as seen, updates their latest and peak CPU/RAM fields, creates entries for new process identities, and emits EXITED events for entries that were not seen. History is bounded to prevent the monitoring process from growing indefinitely during long execution.

Open Applications

The Open Applications feature is intentionally separate from the low-level process table. On native Linux, a process is considered a GUI-session candidate when its environment contains `DISPLAY` or `WAYLAND_DISPLAY` and its memory maps show GUI-library evidence such as X11/XCB, Wayland, GTK/GDK, or Qt GUI libraries. Related process rows can then be grouped under a user-friendly application name.

On WSL, native Windows applications are not present in Linux `/proc`. The project therefore detects WSL and launches a fixed Windows PowerShell Get-Process query using `fork()/exec`. Only processes with visible top-level windows and non-empty window titles are returned. The response includes Windows PID, working-set memory, interval CPU percentage, process name, and visible window title. Browser titles can provide a hint such as “ChatGPT - Google Chrome”, but the project does not claim access to every hidden tab or URL.

The latest remote implementation extends the same concept through authorized SSH. If the remote source is WSL and Windows PowerShell interop is available, the fixed remote script emits visible native Windows application records in addition to Linux/WSL GUI-process records. The frontend labels the operating-system source so Windows-host rows are not confused with Linux PIDs.

HTTP server and API

The C HTTP server uses direct POSIX sockets. It enforces a bounded request size and body size, applies receive/send timeouts, parses only the request information required by the project, and closes each connection after sending a response. Static files are served only from known frontend paths, with path traversal checks for the assets directory. API routes are passed to `api_handle()`, which returns a status code, content type, and dynamically allocated response body.

The server creates a detached pthread for each accepted client. An active-client counter is maintained so shutdown can allow workers a short time to finish. The server defaults to loopback binding so the dashboard is not unintentionally exposed to the entire network. A separate `--bind-lan` option exists for deliberate trusted-LAN use.

Remote SSH collector

Remote collection is performed by the main machine without installing the project on the remote laptop. The backend forks and execs OpenSSH with a fixed list of options, including BatchMode behavior and bounded timeouts. A fixed read-only shell script is written to the SSH process through standard input. The script reads remote `/proc` and `/sys` files and prints normalized records that are parsed locally.

The browser-controlled host value is validated before OpenSSH is invoked. The implementation does not construct a shell command such as `sh -c` from the entered value. SSH username/key selection remains under backend/OpenSSH control. This design gives the project a useful remote feature while preserving normal operating-system authorization boundaries.

6.2 Algorithm / Pseudocode

Algorithm A – Local system snapshot

INPUT: previous snapshot (optional)
OUTPUT: current MonitorSnapshot

1. Read aggregate CPU counters from `/proc/stat`.
2. Read CPU model/core count, memory, uptime and load.
3. Read root filesystem usage with `statvfs()`.
4. Read network interfaces and byte counters.
5. If a previous sample exists, calculate interval CPU and network rates.
6. Scan `/proc/[0-9]*` and parse each process.
7. Match each process with previous PID + `start_ticks`.
8. Calculate per-process CPU percentage and RAM fields.
9. Set source metadata, timestamp and freshness state.
10. Return the normalized snapshot.

Algorithm B – Process lifecycle tracking

```
FOR each tracked entry:
    seen_this_round = false

FOR each process P in current snapshot:
    key = (P.pid, P.start_ticks)
    IF key already tracked:
        mark seen
        update latest/peak CPU and RAM
        optionally emit ACTIVE
    ELSE:
        create tracked entry
        emit STARTED

FOR each tracked entry not seen:
    emit EXITED
    remove/recycle tracked entry
```

Algorithm C – CPU utilization

```
system_delta = current_total_ticks - previous_total_ticks
idle_delta = current_idle_ticks - previous_idle_ticks
CPU_percent = 100 * (system_delta - idle_delta) / system_delta

process_delta = current(utime + stime) - previous(utime + stime)
process_CPU_percent = (process_delta / system_delta) * cpu_core_count * 100
```

Algorithm D – Remote quick connect

1. Receive IPv4 address from POST /api/quick-connect.
2. Reject invalid or injection-like input before any external execution.
3. Resolve backend-only SSH username/port/key defaults.
4. Test authorized SSH access to the remote Linux /proc environment.
5. If successful, create/reuse machine configuration and select it.
6. Reset lifecycle tracking for the new source.
7. Collector obtains remote snapshots through fixed SSH scripts.
8. If collection fails, report remote unavailable/disconnected; do not reuse local values.

6.3 Core C / C++ Implementation

The final project uses C rather than C++. The implementation is divided into source files with narrow responsibilities and header files that define shared structures and public functions. This structure makes it possible to explain the project file-by-file during evaluation.

Table 6.1 – Core C data structures

Core structure	Selected fields / role
SystemInfo	CPU %, load, memory/swap, disk, uptime, process counts, network totals/rates and interfaces.
ProcessInfo	PID, PPID, UID/user, name, state, CPU %, RAM, threads, command line, start ticks/time.
MonitorSnapshot	Source type, machine identity/status, timestamp/age, SystemInfo and process array.
LifecycleEvent	Event type, PID/PPID, timestamps, CPU/RAM, peaks, lifetime and final state.
TrackedProcess	Bounded internal lifecycle state keyed by PID + start_ticks.

Core structure	Selected fields / role
MonitorContext	Mutex, worker thread, snapshots, lifecycle history, selected source and callback state.

Representative monitor-core public interface

<pre>int monitor_init(MonitorContext *ctx, unsigned interval_ms); int monitor_start(MonitorContext *ctx); void monitor_stop(MonitorContext *ctx); int monitor_set_local(MonitorContext *ctx); int monitor_set_remote(MonitorContext *ctx, const char *machine_id, const char *machine_name, monitor_remote_collect_fn cb, void *opaque);</pre>
--

This interface shows that the monitor core is not tied to the browser. The same MonitorContext can be started for the terminal or web program, and the source can be changed between local collection and a remote callback. This modularity is one reason the project can support a terminal mode, web mode, and remote mode without duplicating the entire monitoring logic.

6.4 Linux System Calls / APIs Used

Most kernel information is obtained through read-only virtual files. The project uses bounded helper functions to prevent unbounded reads. When scanning /proc, a process may exit between directory enumeration and file access; therefore, failures for individual PIDs are treated as normal and the scan continues. This is essential because /proc is a dynamic view rather than a static directory.

The project also demonstrates link-oriented process interfaces. /proc/<PID>/exe and /proc/<PID>/cwd are symbolic links, so readlink() is used rather than reading them as regular text files. File-descriptor information is represented as entries in /proc/<PID>/fd. Memory maps are read from /proc/<PID>/maps only on demand.

Table 6.2 – Linux information sources

Interface	Information obtained
/proc/stat	Aggregate CPU tick counters.
/proc/cpuinfo	CPU model and logical processors.
/proc/meminfo	Physical memory and swap information.
/proc/uptime	System/WSL uptime.
/proc/loadavg	1, 5 and 15 minute load averages.
/proc/<PID>/stat	PPID, state, CPU ticks, start time and other process counters.
/proc/<PID>/status	UID, memory and thread fields.
/proc/<PID>/cmdline	Command-line arguments.
/proc/<PID>/io	Per-process I/O counters when permitted.
/proc/<PID>/fd	Open-file-descriptor entries.
/proc/<PID>/maps	Virtual-memory map when permitted.
/sys/class/net	Network link state and byte counters.
statvfs()	Filesystem total/free/used capacity.
getifaddrs()	IPv4 address association for interfaces.

6.5 Process / Thread Implementation

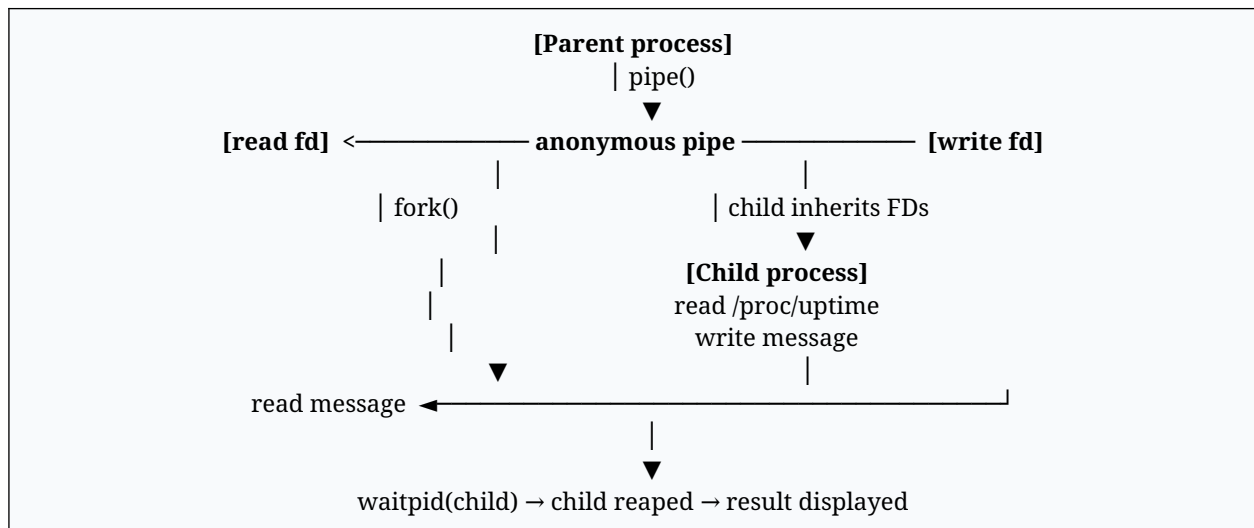
The project uses processes for demonstrations and for launching fixed external system programs, while threads are used for in-process concurrency. The monitor worker is a joinable thread so the main program can request stop and wait for it during shutdown. HTTP client threads are detached because they are short-lived connection handlers; the server maintains a count so it can report if workers are still finishing during shutdown.

Child processes created for fixed probes redirect their output to pipes when the parent needs to capture it. The child then uses `execv()/execvp()` so the intended external program replaces the child image. The parent reads the result with a bounded timeout and calls `waitpid()` to reap the child. This pattern is used instead of `system()` or shell concatenation because it gives the program more direct control over arguments and reduces shell-injection risk.

6.6 IPC / Synchronization Implementation

The fixed process demonstration is the clearest IPC example in the project. The implementation creates a pipe before `fork()` so both processes inherit the descriptors. The child closes the read end, reads `/proc/uptime`, writes the result to the pipe, closes its descriptor, and exits. The parent closes the write end, reads the message, waits for the child with `waitpid()`, and packages the result for terminal or JSON presentation.

Figure 6.1 – Pipe/fork/waitpid IPC demonstration



Thread synchronization is separate from process IPC. A pthread mutex is used to protect monitor state shared between the collector and HTTP/API readers. The mutex is not used as a substitute for `waitpid()`; the project therefore demonstrates two distinct synchronization domains: inter-process synchronization through process-control/IPC mechanisms and intra-process synchronization through pthread primitives.

6.7 Memory Management Implementation

The monitor uses bounded arrays for processes, lifecycle history, network interfaces, and tracked-process entries. This establishes predictable upper bounds for long-running monitoring and avoids uncontrolled growth in the core state. JSON response construction uses dynamically allocated buffers that grow as needed and are freed after the response is copied/sent. File-content helpers also use explicit maximum sizes.

The project treats memory-management correctness as part of systems programming. Error paths free

allocated buffers and close file descriptors. The debug build uses AddressSanitizer and UndefinedBehaviorSanitizer. During the documented sanitizer verification, the one-shot terminal mode, IPC demo, web API, process inspection, application endpoint, and clean shutdown were exercised without sanitizer diagnostics on those tested paths.

The application monitors memory without trying to modify another process address space. Per-process memory maps and status information are read through `/proc` according to Linux permissions. This is consistent with the project's read-only observability objective.

6.8 Optional User Interface Implementation

The web interface, branded KernelScope, is an optional presentation layer over the C core. It uses a dark observability-style layout with a fixed navigation sidebar and pages for Overview, Processes, Lifecycle, Network, Systems Lab, and Machines. The Overview page contains resource cards for CPU, memory, disk, processes, network receive/transmit, swap, and load average, followed by live-resource and machine-detail sections.

The Processes page contains both the Open Applications layer and the raw process table. This distinction is important because an application such as Chrome may correspond to multiple processes, while the raw table must continue to represent real kernel PIDs. A selected process can open a detailed inspector. The Lifecycle page shows process STARTED/ACTIVE/EXITED history, and the Systems Lab page runs the fixed IPC demonstration.

The Machines page exposes the IP-only remote connection flow. Source labels and availability states are shown throughout the dashboard so the user can see whether the values are local, remote, live, stale, or unavailable. The JavaScript polling logic does not calculate authoritative system values; it only requests JSON and renders the backend results.

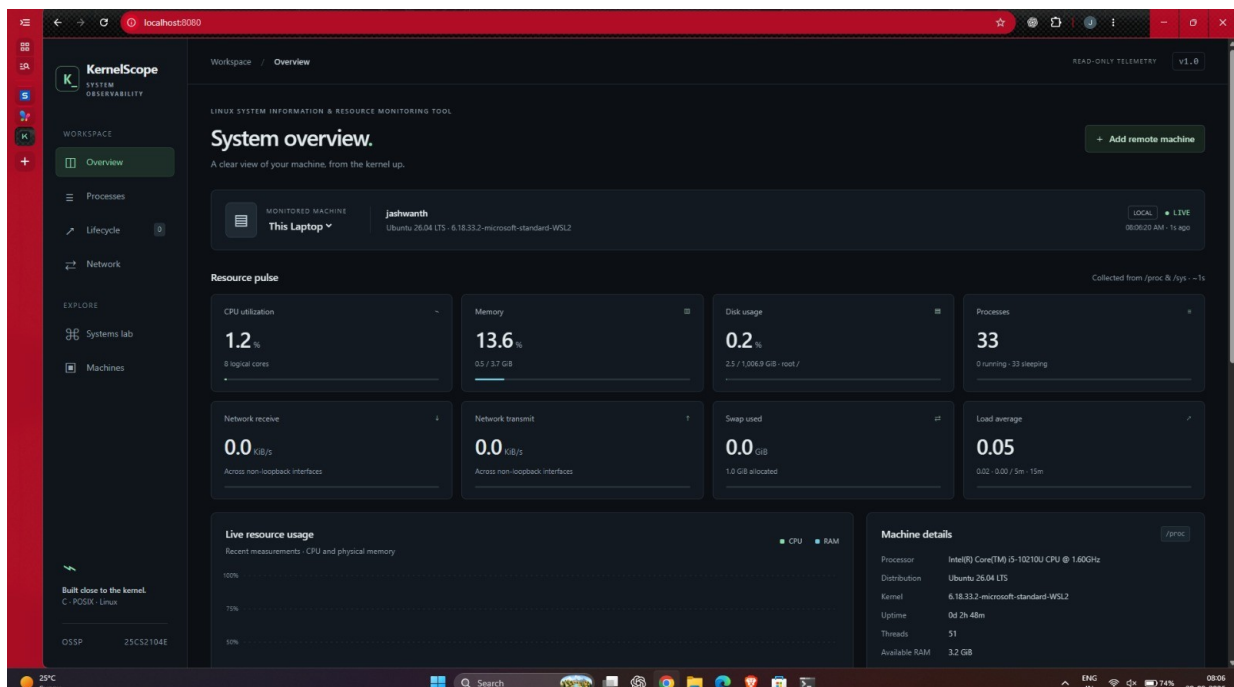


Figure 6.2 – KernelScope overview dashboard during local execution

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled from the Ubuntu/WSL terminal. The source root contains a Makefile that builds the terminal and web binaries from the common monitoring objects and their interface-specific objects. A clean build is recommended before the final review so that the evaluator can see that the executables are produced from source.

Release build commands

```
cd /path/to/Linux_System_Information_and_Resource_Monitoring_Tool
make clean
make
```

For the current finalized project package, the documentation build process re-ran make clean and make with GCC using -std=c11 -Wall -Wextra -Wpedantic -O2 -pthread. Both system_monitor and monitor_web compiled successfully, and no warning lines were produced by the configured warning options.

7.2 Execution Commands

Web dashboard

```
./monitor_web
# Browser: http://localhost:8080

# Optional alternate port
./monitor_web --port 8081

# Deliberate trusted-LAN binding
./monitor_web --bind-lan
```

Terminal monitor

```
./system_monitor
./system_monitor --once
./system_monitor --interval 2
./system_monitor --process-demo
```

Remote monitoring preparation

```
# one-time authorization on the main machine
ssh-keygen -t ed25519
ssh-copy-id remote_user@REMOTE_IP
ssh -o BatchMode=yes remote_user@REMOTE_IP "hostname && cat /proc/uptime"

# backend-only defaults when needed
export MONITOR_SSH_USER="remote_user"
unset MONITOR_SSH_IDENTITY # allow normal OpenSSH key selection
./monitor_web
```

The final BatchMode SSH test is useful before opening the dashboard. If it prints the remote hostname and /proc/uptime without asking for a password, the operating-system authorization is ready. If the remote WSL distribution or SSH service is stopped, the dashboard should show the remote source as unavailable/disconnected.

7.3 Sample Inputs and Outputs

The project is primarily telemetry-driven, so most input is the current operating-system state rather than typed data. Representative output from the API is shown below. Exact numeric values naturally change on every machine and every sampling interval; therefore, the documentation focuses on field meaning rather than treating one measurement as a fixed expected value.

Sample health response

```
GET /api/health
{
  "ok": true,
  "service": "monitor_web"
}
```

Representative system-snapshot output

```
GET /api/system (representative structure)
{
  "source": "local",
  "machine_name": "This Laptop",
  "status": "LOCAL — LIVE",
  "hostname": "<runtime-hostname>",
  "distribution": "<runtime-linux-distribution>",
  "kernel": "<runtime-kernel>",
  "cpu_cores": <number>,
  "cpu_percent": <measured-value>,
  "load": [<1m>, <5m>, <15m>],
  "uptime_seconds": <measured-value>,
  "memory": { "total_kb": ..., "used_kb": ..., "percent": ... },
  "disk": { "total_bytes": ..., "used_bytes": ..., "percent": ... },
  "processes": { "total": ..., "running": ..., "sleeping": ..., "threads": ... },
  "network": { "rx_rate_bps": ..., "tx_rate_bps": ..., "interfaces": [...] }
}
```

The current finalized source was also executed with the supplied automated test suite using `make test`. The tests observed a real temporary process with non-zero CPU utilization near one fully occupied core, detected its STARTED and EXITED lifecycle events, validated application endpoints, and completed the source-isolation, security, and quick-connect checks. The final test runner printed ALL TESTS PASSED.

Table 7.1 – Final automated test summary

Test area	Observed result from current finalized source
Smoke test	PASS – binaries execute and one-shot collection succeeds.
API test	PASS – numeric system values and real PID data returned.
Process test	PASS – live process observed; bounded busy process measured real non-zero CPU.
Lifecycle test	PASS – temporary process generated STARTED and EXITED events.
Applications test	PASS – native application endpoint and WSL Windows visible-app parser path.
Security test	PASS – unsafe/injection-style remote host input rejected.
Machine-state test	PASS – unavailable remote state did not leak the previous local snapshot.
Quick-connect test	PASS – IP-only backend flow and fixed OpenSSH argument behavior.

7.4 Screenshots of Execution

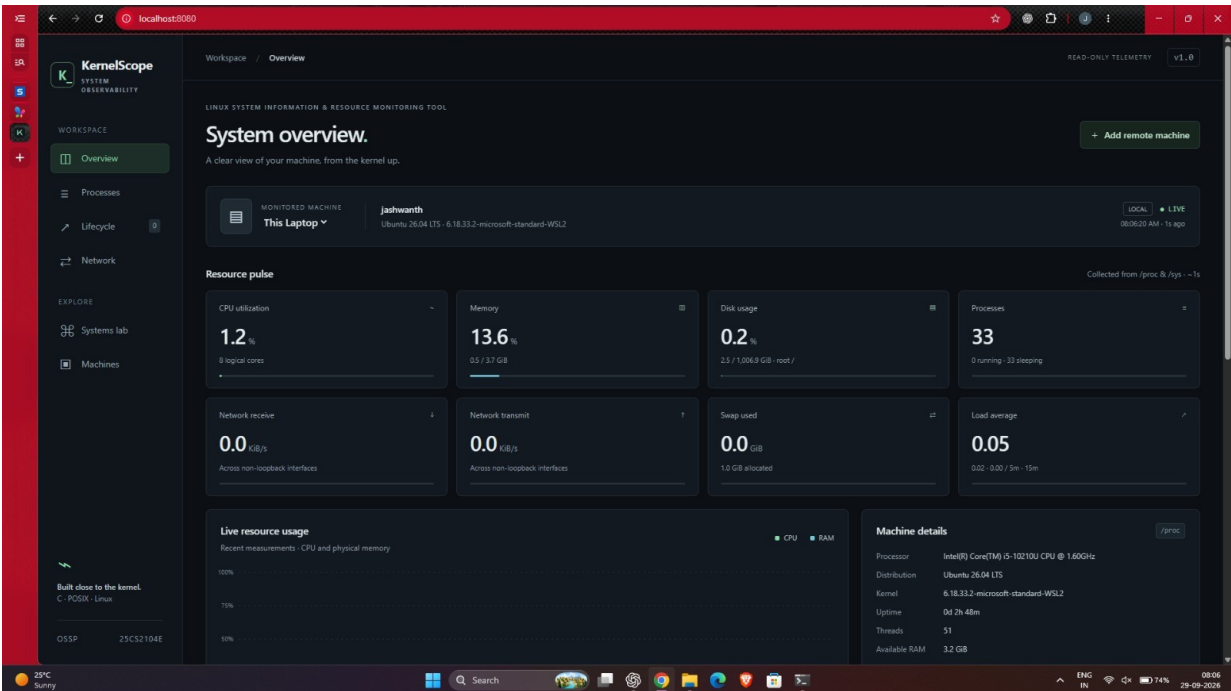


Figure 7.1 – System Overview page showing live Linux/WSL resource information

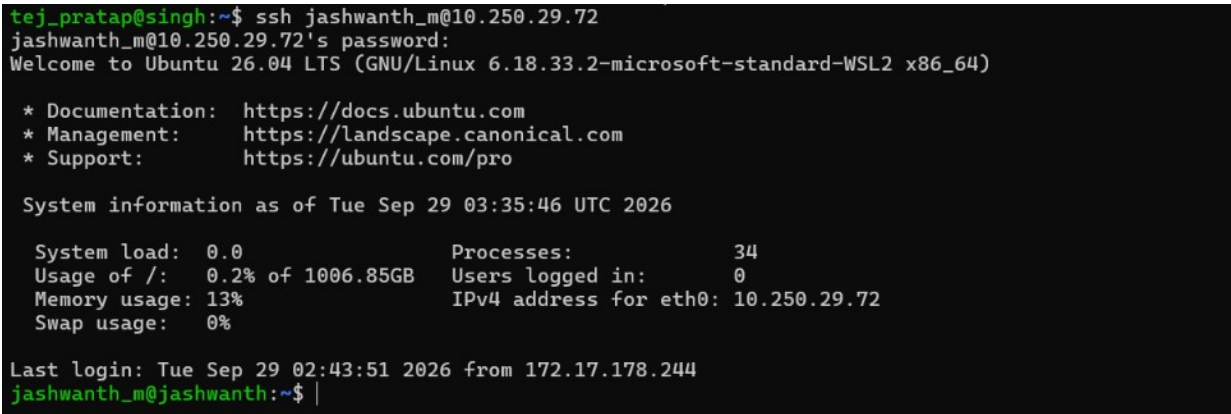


Figure 7.2 – Authorized SSH login to the remote WSL/Linux environment during connectivity validation

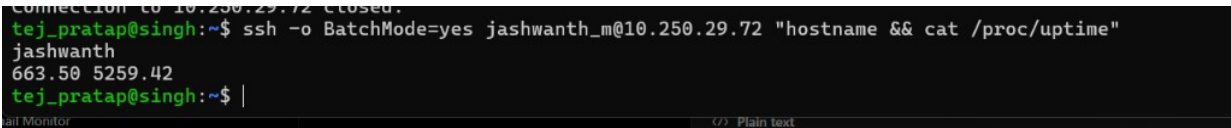


Figure 7.3 – BatchMode SSH validation returning remote hostname and /proc/uptime

Figure 7.4 – Processes and Open Applications page

[FIGURE PLACEHOLDER]
Insert the final screenshot showing the Open Applications cards and the raw Linux process table after running the latest project build.

Figure 7.5 – Process Lifecycle page

[FIGURE PLACEHOLDER]
Insert the final screenshot showing STARTED, ACTIVE, and EXITED events for a short-lived demonstration process.

Figure 7.6 – Remote Windows visible applications through remote WSL

[FIGURE PLACEHOLDER]

Insert the final screenshot showing Chrome/Edge/VS Code or another visible Windows-host application from the authorized remote WSL connection, with the operating-system source label visible.

7.5 Demonstration of OS Concepts

A recommended final-review demonstration starts by launching monitor_web and opening the Overview page. CPU, memory, disk, network, process count, load, and uptime can be related to their kernel sources. The evaluator can then open the Processes page and compare PID, PPID, state, CPU, RAM, and threads with the process concepts taught in class.

Next, a temporary workload can be started so that the lifecycle page records a STARTED event, an ACTIVE period, and an EXITED event. The process inspector can be opened while the process is alive to show command line, executable, working directory, I/O, file descriptors, and memory information. This is an effective way to connect process theory with real /proc entries.

The Systems Lab process demonstration should then be executed. The student can explain that pipe() is created before fork(), the child writes a real /proc/uptime message, the parent reads it, and waitpid() reaps the child. This single demonstration covers process creation, IPC, synchronization, termination, file descriptors, and system programming in C.

Finally, remote monitoring can be demonstrated after the authorized SSH test succeeds. The remote machine is selected from the dashboard using its IP address. The presenter should explicitly explain that the remote Linux values come from the remote /proc and /sys environment and that, on remote WSL, visible Windows applications are obtained separately through controlled PowerShell interop. This distinction demonstrates that the project understands the boundary between the WSL Linux environment and the Windows host.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The Linux System Information and Resource Monitoring Tool is a C-based systems-programming project that combines real Linux telemetry, process observation, lifecycle tracking, IPC demonstration, concurrency, socket programming, a web interface, and authorized remote monitoring. The project does not treat the dashboard as the primary technical contribution; the main contribution is the monitoring core that reads kernel interfaces and exposes them in a structured form.

The final design supports both local and remote Linux/WSL observation. It maintains source correctness, uses ordinary OpenSSH authorization, and separates WSL Linux process data from optional Windows-host measurements. The result is a project that can be demonstrated visually while still allowing every major feature to be explained using operating-system concepts and source files.

8.2 Objectives Achieved

The project achieved the planned objective of collecting real system information without using fabricated metrics. CPU, memory, swap, disk, load, uptime, network, process, and thread information are obtained from the running environment. Per-process inspection and lifecycle tracking extend the project beyond a simple system-information screen.

The project also achieved its systems-programming objectives. File I/O, /proc and /sys, POSIX threads, mutexes, fork(), pipe(), waitpid(), signals, sockets, and exec-based external program invocation are all used in working implementation paths. The optional user interface and remote extension are built on top of this core rather than replacing it.

The remote requirement was implemented as an authorized agentless SSH design. A validated IP-only browser flow is supported, while credentials remain outside the browser. The later project upgrade also added remote WSL host-application collection so that visible Windows applications on an authorized remote Windows/WSL laptop can be shown separately from its Linux process table.

8.3 Results Achieved

The current finalized source was rebuilt during preparation of this documentation using make clean and make. Both binaries compiled successfully using the configured GCC warning flags. The supplied automated suite was then executed with make test and reported PASS for smoke, API, process, lifecycle, applications, security, machine-state, and quick-connect tests, followed by ALL TESTS PASSED.

The process test observed a real process and measured a non-zero per-process CPU percentage using interval jiffy calculations. The lifecycle test detected both the start and exit of a temporary process. Machine-state testing verified that an unavailable remote source returns an unavailable condition rather than leaking the previous local snapshot. Security testing rejected unsafe host input before external execution. These results support the core claims made in this report.

Physical remote demonstrations remain dependent on the environment: the remote Linux/WSL distribution must be running, the SSH server must be active, the network path must allow TCP port 22, and the main machine must have valid authorization. In the project development discussion, passwordless BatchMode SSH was successfully configured on the actual laptops and returned the remote hostname and /proc/uptime, demonstrating that the project can operate across two authorized machines when those environment conditions are satisfied.

8.4 Skills and Learning Outcomes

The project strengthened practical understanding of Linux kernel interfaces, user-space programming,

process control, file descriptors, CPU accounting, memory reporting, concurrency, mutex protection, signals, socket programming, HTTP basics, JSON serialization, SSH authorization, and defensive input validation. It also demonstrated the importance of distinguishing raw operating-system data from presentation-layer interpretation.

From a software-engineering perspective, the project provided experience with modular C design, build automation through Makefiles, shell-based test automation, source control, runtime debugging, sanitizer builds, and documentation. From a presentation perspective, the KernelScope dashboard made it possible to explain low-level OS behavior using a visual sequence that an evaluator can follow in real time.

CHAPTER -9 REFERENCES

9.1 Books

[TO BE ADDED: No specific textbook citation was provided in the project materials. Insert the exact Operating Systems / Systems Programming textbook prescribed by the course if the college requires a book reference.]

9.2 Research Papers

[TO BE ADDED: No research-paper citation was provided or used as a claimed basis for the implementation. Do not insert an unrelated paper only to fill this section.]

9.3 Linux / GNU Documentation

Linux manual pages used for the implemented interfaces: `proc(5)`, `sysfs(5)`, `statvfs(3)`, `getifaddrs(3)`, `fork(2)`, `pipe(2)`, `waitpid(2)`, `sigaction(2)`, `socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `pthread(7)`, `pthread_create(3)`, and `pthread_mutex_lock(3)`.

GNU Compiler Collection (GCC) documentation for C compilation, warning options, optimization, and sanitizer-enabled development builds.

OpenSSH client/server documentation for key-based authorization, BatchMode operation, SSH configuration, and remote command transport.

9.4 Online Resources

Project GitHub repository: https://github.com/Tejpratap-069/KLH-CSE-2026-2027-Team-7-Linux_System_Information_and_Resource_Monitoring_Tool/tree/main/Project

Project source documentation included in the repository: `README.md`, `START_HERE.md`, `docs/ARCHITECTURE.md`, `docs/API.md`, `docs/REMOTE_MONITORING.md`, `docs/WSL_NOTES.md`, `docs/OPEN_APPLICATIONS.md`, `docs/TEST_RESULTS.md`, and `docs/OSSP_CO_MAPPING.md`.

Ubuntu / WSL system documentation used operationally during setup and testing. [TO BE ADDED: exact college-required URL format, if required by the submission rubric.]